
MIBEL

Energy Analytics Platform

Relatório Técnico Final

Mestrado em Engenharia Informática
Tecnologias e Engenharia de Dados (TEAD 2.0)

ESTG, Instituto Politécnico do Porto

Avelino Almeida 8220800@estg.ipp.pt

Fernando Lucas 8030078@estg.ipp.pt

Ano Letivo 2025–2026

Conteúdo

1	Contexto e Cenário	1
1.1	Enquadramento do Mercado Energético Ibérico	1
1.2	Cenário Criado pelo Grupo	1
1.3	Fontes de Dados	1
2	A) Especificação de Data Products	2
2.1	Os Três Data Products	2
2.2	Contratos e SLAs	2
2.3	Decisão de Impacto: <code>feature_set_version</code> no DP-03	2
3	B) Implementação do Lakehouse	3
3.1	Stack Tecnológico e Justificação das Escolhas	3
3.2	Arquitetura Medallion: Bronze, Silver e Gold	3
3.3	Decisão de Impacto: Hive para Bronze, Iceberg para Silver e Gold	3
4	C) Pipelines com Orquestração	4
4.1	Runners Python Idempotentes	4
4.2	DP-02 Streaming: Pipeline de Ingestão Contínua	4
4.3	Auditoria e Lineage: DP-02 Streaming	5
4.4	Orquestração Flyte	5
4.5	Decisão de Impacto: Reprocessamento Incremental com DELETE+INSERT	6
5	D) Serving	6
5.1	Grafana como Camada de Serving	6
5.2	Conteúdo dos Dashboards	6
5.3	Decisão de Impacto: Grafana sobre Trino como Única Camada de Serving	7
6	E) Qualidade e Observabilidade	7
6.1	Filosofia de Quality Gates Bloqueantes	7
6.2	Cobertura de Checks	8
6.3	Anomalias Esperadas e Conhecimento de Domínio (DP-02)	8
6.4	Demonstração de Qualidade de Dados: DP-03	8
6.5	Observabilidade via Grafana	10
6.6	Decisão de Impacto: Quality Gate Bloqueante vs. Apenas Alertar	10
7	F) Machine Learning	10
7.1	Modelos e Feature Tables	10
7.2	Rastreabilidade Completa de Dados por Versão de Modelo	11
7.3	Decisão de Impacto: Split Temporal e Feature Tables Pré-computadas	11
8	Relevância para o Negócio	11
8.1	Impacto Hipotético dos Três Data Products	12
8.1.1	DP-01: Monitorização de Défice Energético	12
8.1.2	DP-02: Estimativa de Custo Horário e Previsão de Carga	12
8.1.3	DP-03: Previsão Meteo → Produção → Preço	12
8.2	Valor da Qualidade como Diferenciador	12
9	Conclusão	12

Contexto e Cenário

Enquadramento do Mercado Energético Ibérico

O MIBEL (*Mercado Ibérico de Electricidade*) é o mercado grossista de electricidade da Península Ibérica, criado em 2007 como resultado de um acordo bilateral entre Portugal e Espanha com o objectivo de harmonizar os mercados eléctricos dos dois países. Em Portugal, a gestão da rede de transporte é da responsabilidade da REN (Rede Energética Nacional), entidade que publica dados horários de produção e consumo, enquanto os preços *day-ahead* são determinados diariamente na bolsa de energia OMIE (Operador do Mercado Ibérico de Energia).

A estrutura de preços do MIBEL é fortemente influenciada pela componente renovável: Portugal dispõe de uma capacidade instalada significativa em energia hídrica, eólica e fotovoltaica, o que torna os preços *spot* particularmente sensíveis às condições meteorológicas. Em períodos de elevada produção renovável e baixa procura, é frequente observar-se preços negativos, fenómeno que o presente projecto captura e documenta explicitamente nos quality gates. A volatilidade dos preços, a crescente penetração de produção renovável e a dependência de condições meteorológicas criam, em conjunto, um ambiente de elevada complexidade analítica que justifica a construção de uma plataforma de dados dedicada.

Cenário Criado pelo Grupo

O grupo definiu um cenário hipotético realista, inspirado nas necessidades analíticas de um comercializador de energia no mercado ibérico. Neste cenário, uma equipa de **Data Engineering e ML** tem como missão construir e operar uma plataforma de dados que suporte três casos de uso distintos:

1. **Monitorizar** em tempo quase real o equilíbrio entre produção e consumo nacionais, com geração de alertas quando o país se aproxima de uma situação de défice energético;
2. **Estimar** o custo horário da energia consumida com base no preço *day-ahead* OMIE, combinando dados de consumo (REN) com preços de mercado (OMIE) para calcular o custo real de cada megawatt-hora consumido;
3. **Prever** a produção eléctrica diária e o preço *spot* a partir de variáveis meteorológicas, permitindo ao departamento comercial antecipar decisões de compra e venda de energia com até sete dias de antecedência.

Para responder a este cenário de forma integrada, o grupo construiu a plataforma **MIBEL Energy Analytics Platform**: um lakehouse local completo, que processa dados históricos desde 2022, implementado com três Data Products independentes, cinco modelos de ML em produção e dashboards operacionais autoprovisionados. A plataforma corre integralmente em Docker Compose, sem dependências de serviços cloud externos, o que garante reprodutibilidade total e independência de custos de infra-estrutura.

Fontes de Dados

A seleção das fontes de dados foi feita com base na sua disponibilidade pública, fiabilidade e alinhamento com os casos de uso definidos. Foram identificadas três fontes primárias:

Fonte	Data Product	Descrição
REN/ERSE CSV	DP-01	Produção (DGM + PRE) e consumo nacional a 15 min; dados públicos disponíveis em ren.pt
OMIE CSV	DP-02	Preços <i>day-ahead</i> hora a hora (€/MWh); cobrindo o mercado ibérico desde 2022
Open-Meteo API	DP-03	Dados meteorológicos horários para Portugal continental; API REST gratuita com histórico reanálise ERA5
Energy-Charts API	DP-02 Streaming	Consumo e produção via API pública da Fraunhofer ISE; usado como alternativa ao ENTSO-E sem necessidade de token

A opção pela Open-Meteo como fonte meteorológica no DP-03 deveu-se à combinação de acesso

gratuito sem autenticação, cobertura histórica desde 1940 via reanálise ERA5 e disponibilidade de previsões com horizonte de 7 dias, fundamental para o caso de uso de *hedging* antecipado.

A) Especificação de Data Products

Os Três Data Products

A especificação dos Data Products foi o ponto de partida do projecto, antes de qualquer implementação técnica. O grupo começou por definir os contratos de dados, as perguntas analíticas que cada produto deve responder e os SLAs operacionais, alinhando-os com o cenário de negócio descrito anteriormente. Esta abordagem *data product first* garantiu que as escolhas técnicas posteriores (formato de tabela, granularidade, chaves de particionamento) servem os consumidores finais e não o inverso.

Foram especificados três produtos de dados com contratos formais, SLAs/SLOs e estratégia de versionamento semântico:

#	Nome	Grão	Gold Table
DP-01	producao_consumo	Horário	dp_energia_balance_hourly
DP-02	consumo_preco	Horário	dp_energy_market_hourly
DP-03	meteo_producao	Diário	dp_meteo_producao_daily_features

O grão horário para DP-01 e DP-02 resulta de uma decisão de agregação deliberada: os dados brutos da REN chegam a 15 minutos, mas o preço OMIE é publicado por hora. A agregação para hora na camada Silver permite o join natural entre consumo e preço sem necessidade de interpolação.

Contratos e SLAs

Cada Data Product define um contrato formal que inclui: as perguntas analíticas que o produto responde, as métricas de negócio que calcula, o grão e a chave primária, o schema tipado com versão (v1), as regras de qualidade e os SLAs de *freshness* e completude. O contrato funciona como uma interface pública: os consumidores (dashboards, modelos ML) dependem do schema v1 e os produtores (pipelines ETL) comprometem-se a respeitá-lo.

Exemplo de SLAs definidos para o DP-01:

- *Freshness*: dados disponíveis até T+30 min após fecho da hora
- *Completeness*: $\geq 99,0\%$ das horas no intervalo esperado
- *Unicidade*: chave `timestamp_utc` sem duplicados
- *Qualidade*: `flag_defice` e `flag_excedente` mutuamente exclusivos

Os SLAs foram calibrados com base nas características reais das fontes: a REN publica dados com atraso de 15–30 minutos, pelo que o SLA de T+30min é alcançável em operação normal. A completude de 99% tolera até ~88 horas em falta por ano, margem adequada para manutenções planeadas e falhas pontuais de API.

Decisão de Impacto: `feature_set_version` no DP-03

O DP-03 inclui a coluna `feature_set_version` (obrigatória, não nula) em todas as linhas da tabela Gold. Esta decisão surgiu de um problema recorrente em projectos ML: quando as *features* de uma tabela evoluem (nova variável meteorológica, mudança de janela temporal de agregação), é impossível determinar com que versão de features um modelo registado no MLflow foi treinado, tornando a reprodutibilidade impossível.

A solução implementada garante que cada *run* de treino referencia explicitamente a versão do conjunto de *features* utilizado, com as seguintes consequências práticas:

- **Reprodutibilidade**: novo treino com dados históricos da mesma `feature_set_version` produz exactamente o mesmo conjunto de *features* de entrada, independentemente de alterações posteriores à tabela Gold

- **Rastreabilidade:** o registo MLflow associa `feature_set_version` a `dataset=iceberg.gold.<tabela>`, permitindo auditar que dados alimentaram cada modelo em produção
- **Evolução segura:** uma nova transformação de *feature* implica aumentar o número de versão major, sem quebrar consumidores que ainda referenciam a versão anterior

B) Implementação do Lakehouse

Stack Tecnológico e Justificação das Escolhas

O lakehouse corre integralmente em Docker Compose, sem dependências de serviços cloud externos. Esta escolha foi deliberada: o objectivo académico exige reprodutibilidade total (qualquer máquina com Docker replica o ambiente completo) e independência de custos de infra-estrutura cloud. Cada serviço foi seleccionado por ser a solução de referência na sua categoria e por integrar nativamente no ecossistema Apache Iceberg / Trino:

Serviço	Tecnologia	Função e justificação da escolha
Object Storage	MinIO	S3-compatível; armazena Parquet Bronze e artefactos ML. Escolhido por ser o standard de-facto para S3-on-premises, compatível com qualquer SDK boto3/S3 sem alteração de código
Metastore	Hive 4.0 + MariaDB	Registo de schemas para catálogos Hive e Iceberg. A versão 4.0 suporta nativamente tabelas Iceberg sem patches adicionais
Query Engine	Trino 440	Federação multi-catálogo (Hive, Iceberg, Kafka); execução de SQL analítico sobre ficheiros Parquet em MinIO. Separação compute/storage permite escalar o motor de query independentemente do lakehouse
ML Tracking	MLflow + Postgres	Registo de experimentos, métricas, modelos e artefactos. Backend Postgres garante persistência após recreação de contentores
Dashboards	Grafana 10.4	Visualização directa sobre Trino via plugin Trino datasources; sem ETL adicional entre Gold e dashboard
Streaming	Redpanda	Broker Kafka-compatível para ingestão em <i>stream</i> do DP-02. Redpanda foi preferido ao Kafka por ter menor footprint de memória e não requerer ZooKeeper

Arquitetura Medallion: Bronze, Silver e Gold

A arquitetura medallion foi adotada por ser o padrão dominante em engenharia de dados moderna, com separação clara de responsabilidades entre camadas. A implementação concreta no MIBEL define o papel de cada camada da seguinte forma:

Bronze

Tabelas Hive *external* sobre ficheiros Parquet em `s3://warehouse/bronze/`. Não há qualquer transformação de dados, a camada Bronze preserva o dado original exactamente como foi recebido da fonte, servindo como ponto de auditoria e reprocessamento. Se uma transformação Silver ou Gold produzir resultados incorretos, é possível reprocessar a partir do Bronze sem voltar a ingerir dados da fonte.

Silver

Tabelas Iceberg *managed* com deduplicação via `ROW_NUMBER()`, validação de *range* físico (limites realistas para Portugal continental) e classificação de cada linha com `_quality_flag`: `ok` / `null_values` / `out_of_range`. A Silver é a camada de confiança: downstream nunca vê dados duplicados nem valores fisicamente impossíveis.

Gold

Tabelas Iceberg *managed* com joins *cross-DP*, lags temporais (1h, 24h, 7d), *rolling windows* e flags de negócio calculados em SQL Trino. A Gold é a única fonte de verdade para dashboards, modelos ML e quality gates, nenhum consumidor acede a Bronze ou Silver directamente.

Decisão de Impacto: Hive para Bronze, Iceberg para Silver e Gold

A separação deliberada entre Hive (Bronze) e Iceberg (Silver/Gold) é uma das decisões de maior impacto arquitetural do projecto. Em vez de usar um único formato de tabela para todas as camadas,

o grupo optou por formatos diferentes com base nas garantias necessárias em cada camada:

- **Iceberg nas camadas Silver e Gold** oferece transações ACID (operações DELETE+INSERT atômicas), *schema evolution* sem rewrite de ficheiros e *time-travel*, funcionalidades essenciais para auditar reprocessamentos, fazer rollback de transformações incorretas e garantir consistência durante escritas concorrentes
- **Hive external na Bronze** elimina o custo de gestão ACID em dados raw, que são escritos uma única vez e nunca modificados. As tabelas Hive *external* apontam directamente para os ficheiros Parquet no MinIO sem overhead de gestão transaccional, resultando em menor latência de ingestão e maior simplicidade operacional
- A combinação das duas tecnologias no mesmo Hive Metastore e no mesmo catálogo Trino permite queries que cruzam Bronze e Silver sem infra-estrutura adicional, o que é útil para debugging e validação de transformações

C) Pipelines com Orquestração

Runners Python Idempotentes

O grupo optou por implementar os orquestradores como scripts Python autónomos em vez de usar um orquestrador externo como Airflow ou Prefect para todos os DPs. Esta decisão foi justificada pela escala do projecto: para três Data Products com execução predominantemente batch, um orquestrador externo adicionaria complexidade operacional sem benefício proporcional. O Flyte foi adotado apenas onde há paralelismo natural entre tarefas (DP-02 Static com múltiplos meses de dados históricos).

Cada orquestrador Python executa as seguintes fases em sequência, com verificações de saúde em cada etapa:

1. Verificação Docker + `compose up` (opcional com `-skip-docker`; o stack pode já estar a correr)
2. *Health check* Trino com `SELECT 1` até resposta positiva (máx. 60s com *retry*)
3. Criação de *venv* local isolado + instalação de dependências Python específicas do DP
4. DDL: `CREATE TABLE IF NOT EXISTS` para Bronze, Silver e Gold (idempotente; novas execuções seguras)
5. Fetch / upload de dados para MinIO (download de CSV da fonte ou chamada Open-Meteo API)
6. Bronze INSERT via Trino (carga de Parquet em tabela Hive *external*)
7. Silver INSERT com deduplicação via `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`, validação de *range* e classificação `_quality_flag`
8. Gold INSERT com joins *cross-DP*, lags temporais e *rolling windows* calculados em SQL
9. Quality Gate automático (bloqueante em FAIL; ver Secção 6)

A idempotência é garantida em todas as fases: DDL usa `IF NOT EXISTS`, a Bronze verifica datas já carregadas antes de re-inserir, e a Gold usa DELETE+INSERT por intervalo de datas (ver subsecção abaixo).

DP-02 Streaming: Pipeline de Ingestão Contínua

O DP-02 tem duas variantes de pipeline que coexistem e servem propósitos complementares, partilhando a mesma camada Gold para garantir consistência analítica:

Static (OMIE CSV)

Carrega dados históricos 2022–2024 via Flyte remoto em pods K3s; cada ficheiro mensal OMIE é processado em paralelo como uma tarefa Flyte independente. Adequado para análise retrospectiva, treino ML com dados históricos extensos e validação de modelos com janelas temporais longas.

Streaming (ENTSO-E / Energy-Charts API)

Ingestão contínua dos últimos 12 meses via API; projetado para execução diária em *cron*, mantendo os dados do dashboard sempre actualizados. O Energy-Charts (Fraunhofer ISE) serve como fonte

primária por não requerer token de autenticação; o ENTSO-E Transparency Platform é suportado como alternativa com maior granularidade.

O orquestrador `run_streaming_pipeline.py` foi desenhado com mecanismos de resiliência que os restantes DPs não necessitam, dado que é o único a correr em modo quase-contínuo:

- **Retry com backoff exponencial:** 3 tentativas automáticas ($2s \rightarrow 4s \rightarrow 8s$) sobre cada chamada de API e cada tarefa Flyte; erros transitórios de rede ou de API não abortam o pipeline, sendo registados e retomados automaticamente
- **Compactação automática Iceberg:** `ALTER TABLE ... EXECUTE optimize` após cada run bem-sucedido; ingestões incrementais diárias geram muitos ficheiros pequenos (*small-files problem*), que degradam a performance de queries analíticas; a compactação automática mantém o tamanho médio dos ficheiros acima de 128 MB
- **SLA operacional monitorizado:** *freshness* máx. 7 dias e *runtime* máx. 45 min; avisos são persistidos em tabela de auditoria e visíveis no dashboard de observabilidade
- **Flags de controlo granulares:** `-source {energycharts|entsoe}`, `-start`, `-end`, `-today`, `-full`, `-clean`, permitindo reprocessamento cirúrgico de intervalos específicos sem voltar a correr o histórico completo

Auditoria e Lineage: DP-02 Streaming

Uma das capacidades mais relevantes do DP-02 Streaming é a persistência automática de metadados de execução em tabelas Iceberg dedicadas no schema `iceberg.audit`. Esta funcionalidade permite correlacionar qualquer dado da Gold com a execução específica que o produziu, fechando o ciclo de rastreabilidade dados $\leftrightarrow$ execução.

Cada execução do pipeline Streaming persiste automaticamente dois registos:

`audit.pipeline_runs`

Uma linha por run com: `run_id` (UUID v4), `status` (SUCCESS/FAILED), `duration_seconds`, contagens de linhas inseridas por camada (`rows_bronze`, `rows_silver`, `rows_gold`), parâmetros de execução (`source`, `date_from`, `date_to`) e mensagem de erro em caso de falha.

`audit.dataset_lineage`

Grafo upstream $\rightarrow$ downstream materializado por execução, documentando a cadeia de transformações: `bronze.consumo_api_raw` $\rightarrow$ `silver.consumo_api_hourly` $\rightarrow$ `gold.dp_energy_market_api_hourly`. Cada aresta do grafo inclui o `run_id`, permitindo reconstruir que runs produziram cada versão dos dados.

O logging estruturado prefixa os primeiros 8 caracteres do UUID em cada linha de log, correlacionando directamente os registos do terminal com as linhas da tabela de auditoria:

```
[3f2a1b9c] Pipeline iniciada - periodo=2024-01-01->2024-12-31  fonte=energycharts
[3f2a1b9c] Bronze ingerido: 17544 linhas (consumo + preco)
[3f2a1b9c] Pipeline SUCCESS em 541s (bronze=17544 silver=8772 gold=8748)
```

Orquestração Flyte

O Flyte foi adotado para o DP-02 Static como motor de orquestração remota, dado que o processamento paralelo de múltiplos meses de dados históricos reduz significativamente o tempo total de carga. O Flyte Sandbox corre como contentor externo com K3s interno, acedendo aos serviços do Compose via `host.docker.internal`, solução necessária porque os pods K3s não partilham a rede Docker Compose.

Os scripts de pipeline são decorados com `@task` e `@workflow` do SDK Flyte, com *stubs* locais que permitem execução sem o Flyte instalado (útil em ambientes de desenvolvimento rápido):

```
try:
    from flytekit import task, workflow
```



```
except ModuleNotFoundError:
    def task(*args, **kwargs):
        def decorator(func): return func
        return args[0] if args and callable(args[0]) else decorator
    def workflow(func=None, **kwargs):
        return func if func else lambda f: f
```

Esta abordagem de *graceful degradation* garante que o código de pipeline funciona em modo local sem Flyte instalado, reduzindo a fricção durante desenvolvimento e debug, e corre em modo orquestrado em K3s quando o Flyte Sandbox está disponível.

Decisão de Impacto: Reprocessamento Incremental com DELETE+INSERT

O DP-01, DP-02 Static e DP-03 suportam reprocessamento incremental da camada Gold via flags `-date-from` / `-date-to`. A estratégia DELETE+INSERT (em vez de UPSERT ou INSERT OVERWRITE) foi escolhida após análise das alternativas:

- **Atomicidade garantida pelo Iceberg:** o DELETE e o INSERT subsequente são commitados numa única transação Iceberg; nunca há um instante em que a Gold tem um intervalo apagado mas ainda não reescrito, eliminando o risco de queries intermediárias lerem dados incompletos
- **Sem necessidade de chave de merge:** UPSERT em Iceberg requer uma chave de negócio bem definida e uma operação MERGE INTO com lógica de deduplicação; DELETE+INSERT elimina esta complexidade porque o intervalo é sempre substituído por completo
- **Isolamento cirúrgico:** o reprocessamento de um mês específico não afeta dados de outros meses; é possível corrigir uma transformação Gold para Janeiro 2024 sem tocar nos restantes meses

D) Serving

Grafana como Camada de Serving

A estratégia de serving do projecto assenta exclusivamente em **Grafana 10.4**, que consulta directamente as tabelas Gold via Trino. Esta abordagem contrasta com uma arquitetura alternativa comum que interpõe uma API REST Python entre o lakehouse e os dashboards. A escolha por Grafana directo sobre Trino foi tomada após comparar as duas opções no contexto do projecto.

Todos os dashboards são autoprovisionados a partir de ficheiros JSON em `04_application/grafana/dashboards/` e carregados automaticamente no arranque do Grafana, sem necessidade de configuração manual. O datasource Trino é configurado via variável de ambiente, apontando para o serviço Trino do Docker Compose.

DP	Dashboard	Gold Table consultada
DP-01	<code>producao_consumo_overview.json</code>	<code>iceberg.gold.dp_energia_balance_hourly</code>
DP-02 Static	<code>consumo_preco_overview.json</code>	<code>iceberg.gold.dp_energy_market_hourly</code>
DP-02 Streaming	<code>consumo_preco_streaming_overview.json</code>	<code>iceberg.gold.dp_energy_market_api_hourly</code>
DP-03	<code>meteo_producao_overview.json</code>	<code>iceberg.gold.dp_meteo_producao_daily_features</code>

Conteúdo dos Dashboards

Cada dashboard foi desenhado com foco no caso de uso de negócio do respetivo DP, não apenas como visualização genérica de dados. Os painéis foram estruturados para responder directamente às perguntas analíticas definidas no contrato de cada Data Product:

- **DP-01:** saldo líquido nacional (MWh = produção – consumo), ratio produção/consumo por hora, contagem de horas de défice e excedente no período seleccionado, série temporal horária com banda de tolerância; o dashboard responde à pergunta “Portugal está em défice energético agora?”

- **DP-02 Static:** preço médio *day-ahead* por mês (€/MWh), custo estimado total da energia consumida, correlação entre preço e consumo, percentagem de horas com preço negativo; o dashboard responde à pergunta “Quanto custou a energia em cada mês e que fatores determinaram esse custo?”
- **DP-02 Streaming:** dados quase-em-tempo-real via ENTSO-E/Energy-Charts, *rolling average* 24h do consumo, comparação Portugal vs. Espanha, indicador de *freshness* com semáforo ligado directamente à tabela `audit.pipeline_runs`
- **DP-03:** temperatura média diária e precipitação acumulada, produção renovável total (hídrica + eólica + solar), *scatter plot* temperatura vs. produção com linha de tendência, preço *spot* estimado pelo Modelo B

Decisão de Impacto: Grafana sobre Trino como Única Camada de Serving

A decisão de usar Grafana directamente sobre Trino, eliminando qualquer backend HTTP intermédio, tem impacto significativo na manutenibilidade e correção do sistema:

- **Eliminação do *serving skew*:** uma API Python intermediária teria de reimplementar parte da lógica de agregação e filtragem já presente na Gold; qualquer divergência entre a lógica da API e a lógica SQL introduz inconsistências silenciosas entre o que os dashboards mostram e o que os modelos ML usam para treinar
- **Dashboards como código versionado:** os ficheiros JSON em git são a única fonte de verdade para os dashboards; qualquer alteração é rastreável, reversível e aplicável a qualquer instância do ambiente sem configuração manual
- **Provisionamento instantâneo:** novos dashboards ou alterações a dashboards existentes são carregados via `POST /api/admin/provisioning/dashboards/reload` sem reiniciar o stack, com tempo de aplicação inferior a 1 segundo
- **Redução da superfície operacional:** a eliminação de três servidores Python (um por DP com API REST) reduz o número de processos a monitorizar, de dependências a gerir e de potenciais pontos de falha

E) Qualidade e Observabilidade

Filosofia de Quality Gates Bloqueantes

A abordagem de qualidade do projecto é diferente do que normalmente se observa em projectos académicos: em vez de validações *best-effort* ou simples logging de anomalias, o grupo implementou quality gates **bloqueantes** que impedem dados de má qualidade de chegar a consumidores downstream. Esta decisão reflecte uma visão de qualidade como propriedade sistémica e não como verificação pontual.

Cada runner executa um quality gate automático após a carga da Gold. O padrão de output é uniforme e persistido em tabela de auditoria:

check_name	status	valor_pct	threshold_pct	detalhe
------------	--------	-----------	---------------	---------

- **PASS:** valor dentro dos limites definidos no contrato do DP
- **WARN:** anomalia não crítica que não viola o contrato mas merece atenção, pipeline continua, log emitido, registo persistido em `quality_log`
- **FAIL:** violação crítica do contrato, com `SystemExit` imediato; os dados corrompidos não chegam a dashboards, modelos ML ou APIs

Todos os resultados de todos os checks são persistidos automaticamente na tabela `iceberg.gold.quality_log` (particionada por `run_date`), permitindo auditoria histórica completa, *trending* de WARNs ao longo do tempo e rastreabilidade de que execução produziu cada resultado de qualidade.

Cobertura de Checks

O grupo implementou 110 checks de qualidade distribuídos por todas as camadas (Bronze, Silver, Gold) de todos os Data Products:

DP	Bronze checks	Silver checks	Gold checks	Total
DP-01	,	11	10	21
DP-02 Static	11	13	14	38
DP-02 Streaming	10	8	8	26
DP-03	,	13	12	25
Total				110

O DP-02 é o mais extensivamente validado ($38 + 26 = 64$ checks) porque combina duas fontes de dados (consumo e preço) com comportamentos distintos, e porque opera em modo quase-contínuo onde anomalias de ingestão são mais prováveis.

Os tipos de verificação implementados cobrem as principais dimensões de qualidade de dados:

- **Null rates:** colunas críticas devem ter 0% de nulos (FAIL) ou $< 5\%$ (WARN); o limiar de 5% WARN foi calibrado para tolerância a lacunas pontuais de API sem bloquear pipelines desnecessariamente
- **Range físico:** temperatura $\in [-10, 50]^\circ\text{C}$, precipitação $\in [0, 200]$ mm, vento $\in [0, 80]$ m/s, limiares definidos com base nas condições climatológicas de Portugal continental registadas no século XX
- **Unicidade:** chaves de negócio (`timestamp_utc`, `data_hora`) sem duplicados na Silver e Gold
- **Alinhamento temporal:** *timestamps* alinhados à hora UTC ou a intervalos de 15 min (dados brutos REN)
- **Lag consistency:** `temp_lag_1d` deve corresponder à temperatura observada no dia anterior (tolerância $0,01^\circ\text{C}$), validando que os lags temporais foram calculados correctamente
- **Cobertura cross-DP:** $\geq 90\%$ dos dias devem ter dados de meteo + produção + preço em simultâneo, garantindo que a Gold do DP-03 tem os três domínios representados
- **Freshness:** dados com atraso < 400 dias (threshold adaptado ao dataset histórico que cobre 2022–2024)

Anomalias Esperadas e Conhecimento de Domínio (DP-02)

Na execução de validação do DP-02 (*consumo_preco*), todos os 38 checks foram executados com resultado: 32 PASS, 6 WARN, 0 FAIL. Os WARNs são documentados e aceites porque representam características conhecidas do mercado energético ibérico ou artefactos controlados do processo de ingestão. Esta distinção entre WARN e FAIL é uma decisão de engenharia deliberada: incluir estas situações como FAIL causaria *alert fatigue* e levaria à ignorância de alertas genuinamente críticos.

Demonstração de Qualidade de Dados: DP-03

A camada Silver do DP-03 (*meteo_producao*) implementa um mecanismo de classificação de qualidade linha a linha: cada registo recebe um `_quality_flag` (ok / `null_values` / `out_of_range`) com base em regras físicas calibradas para Portugal continental. A Gold filtra explicitamente `WHERE _quality_flag = 'ok'`, garantindo que dados inválidos nunca chegam ao pipeline ML nem a dashboards.

Para evidenciar este mecanismo de forma controlada, foi desenvolvido o script `corrupt_bronze.py` (em `01_bronze/data_quality_demo/`), que injeta padrões de sujidade directamente nas tabelas Iceberg Bronze via Trino, sem modificar os scripts de ingestão de produção. A motivação é explícita: os pipelines de API (DP-02 Streaming, DP-03) têm validação mínima na ingestão: nulos são filtrados, mas intervalos físicos e duplicados não são verificados na Bronze. A Silver é, deliberadamente, a única camada de limpeza.

Check	Camada	Contagem	Explicação
price_portugal_raw >= 0	Bronze + Silver	504 horas	Preços negativos são fenómeno real do MIBEL: excesso de produção renovável em períodos de baixa procura empurra o preço abaixo de zero. O dado é válido e preservado; o WARN garante rastreabilidade.
uniqueness (datahora, process_date)	Bronze	24 grupos	Artefacto de re-ingest idempotente: uma segunda execução sobre o mesmo dia gera duplicados temporários na Bronze. A Silver absorve-os via GROUP BY + AVG , sem propagação para camadas superiores.
registos_por_dia >= 80	Bronze	2 dias	Dias de fronteira do ficheiro histórico com carga parcial (< 96 registos de 15 min esperados). Não afecta a agregação horária da Silver porque as horas completas são processadas correctamente.
horas_por_dia >= 23	Silver	2 dias	Dias de transição DST (<i>spring forward</i>): a hora 2 local não existe, pelo que o dia tem 23 horas. Comportamento esperado do calendário europeu de horário de verão.

Tabela 1: WARNs observados no DP-02 e respectiva justificação de domínio.

Tipo	O que injeta	Como Silver responde
nulls	temperature_2m nas primeiras = NULL N linhas	_quality_flag = 'null_values'
outofrange	temp=-50°C, radiation=-200 W/m ² , cloud=150%	_quality_flag = 'out_of_range'
duplicates	Cópias de <i>timestamps</i> com _ingested_at=1970	Silver deduplica via ROW_NUMBEROVERPARTITIONBYts_utcORDERBY_ingested_atDESC , mantendo o registo mais recente
timestamps	<i>Timestamps</i> com minute=30 (não-horários)	Check S12 reporta FAIL em alinhamento temporal

Tabela 2: Padrões de sujidade implementados na demo DP-03 e resposta da camada Silver.

Com injeção de 5% de corrupção (`-type all -pct 5`, ~26 000 linhas Silver), o impacto observado é o seguinte:

- Linhas `ok`: ~95% (~24 700); linhas `null_values`: ~2,5%; linhas `out_of_range`: ~2,5%
- Check S13 (`pct_ok >= 95%`) transita para **WARN** (limiar exactamente na fronteira)
- Checks S06–S10 (intervalos físicos de meteorologia: temperatura, precipitação, vento, radiação, nebulosidade) transitam para **FAIL**, bloqueando a promoção para Gold até ao restauro dos dados originais

O restauro é imediato via `-restore`, que elimina as linhas corrompidas da Bronze. A nova execução da Silver confirma que todos os checks regressam a **PASS** e que a Gold não foi contaminada durante o período de corrupção, uma vez que a Gold filtra explicitamente por `_quality_flag = 'ok'` antes de qualquer join ou cálculo de features.

A demo valida empiricamente o papel de cada camada na arquitetura medallion: a Bronze actua como *landing zone* fiel à fonte (sem rejeições nem transformações), enquanto a Silver é a única fronteira de limpeza que impede dados inválidos de contaminar targets de treino ML e visualizações operacionais.

Observabilidade via Grafana

O ciclo de observabilidade é fechado através de painéis Grafana que consultam as tabelas de auditoria directamente. O DP-02 Streaming expõe as seguintes métricas de pipeline no mesmo dashboard dos dados de negócio:

- Histograma de `duration_seconds` por execução, permitindo detectar degradação de performance ao longo do tempo
- Série temporal de `rows_gold` por run, permite detectar gaps de ingestão (runs com zero linhas inseridas na Gold)
- Painel de *freshness* com semáforo: `CURRENT_DATE - MAX(process_date)` em dias: verde < 3 dias, amarelo 3–7 dias e vermelho > 7 dias
- Tabela das últimas 10 execuções com status, duração e contagens de linhas por camada

Esta integração é especialmente importante porque o mesmo pipeline que produz os dados de negócio produz também os metadados de execução, visíveis no mesmo Grafana sem infra-estrutura adicional de monitorização.

Decisão de Impacto: Quality Gate Bloqueante vs. Apenas Alertar

A decisão de bloquear o pipeline em **FAIL** (em vez de apenas emitir um alerta e continuar) tem impacto directo na qualidade de todo o sistema downstream:

- Um erro de qualidade na Gold propagaria de forma silenciosa para modelos ML (métricas de avaliação inflacionadas por dados corrompidos), para dashboards (valores incorretos visíveis a stakeholders) e para qualquer consumidor externo
- O custo de um **FAIL** na pipeline é baixo e conhecido: corrigir a causa raiz, voltar a correr o pipeline, verificar que os checks passam
- O custo de um modelo ML treinado com dados corrompidos é alto e muitas vezes invisível: o modelo parece funcionar, as métricas parecem boas, mas as previsões em produção são sistematicamente erradas
- A rastreabilidade dados ↔ modelo (via MLflow tags) permite determinar retroativamente se um modelo em produção foi treinado com dados que posteriormente falharam checks de qualidade

F) Machine Learning

Modelos e Feature Tables

Para cada Data Product foram desenvolvidos modelos de ML alinhados com o caso de uso de negócio. A escolha dos algoritmos foi guiada pela natureza do problema (classificação vs. regressão)

e pelas características das séries temporais energéticas (sazonalidade forte, dependência sequencial, não-linearidades):

DP	Modelo	Target	Feature Table (Gold)
DP-01	RF Classifier	flag_defice (t+1h)	dp_energia_balance_hourly
DP-02 Static	GB Regressor	consumo_next_hour	feat_load_forecasting_hourly
DP-02 Streaming	GB Regressor	consumo_next_hour	feat_load_forecasting_api_hourly
DP-03 A	RF Regressor	producao_total_ daily_mwh	dp_meteo_producao_daily_features
DP-03 B	GB Regressor	preco_spot_medio_ eur_mwh	dp_meteo_producao_daily_features

A escolha do **RandomForest** para o DP-01 (classificação de déficit) deve-se à sua robustez com classes desbalanceadas (déficit é um evento raro) e à interpretabilidade via **feature_importances**. O **GradientBoosting** foi preferido para as tarefas de regressão por capturar melhor as não-linearidades e interações entre variáveis que são características dos mercados energéticos.

O DP-02 Streaming tem a sua própria feature table (**feat_load_forecasting_api_hourly**) derivada dos dados ENTSO-E/Energy-Charts, com as mesmas colunas que a variante estática mas particionada por **year/month** para eficiência em ingestões incrementais diárias.

Rastreabilidade Completa de Dados por Versão de Modelo

Cada *run* MLflow regista um conjunto de metadados que permite reconstruir completamente as condições em que o modelo foi treinado:

- **Tags:** **dataset** (tabela Gold exacta, e.g. `iceberg.gold.dp_energia_balance_hourly`), **target**, **model_type**, **domain**
- **Params:** hiperparâmetros, janelas de treino/teste, dimensões dos conjuntos e **random_state**
- **Métricas:** avaliação exclusivamente no conjunto de teste (MAE, RMSE, R^2 , MAPE para regressão; Accuracy, F1 ponderado, ROC-AUC para classificação)
- **Artefactos:** importâncias, colunas de *features* e pipeline sklearn serializado
- **Model Registry:** 5 modelos registados com **registered_model_name**, permitindo comparação de versões e promoção explícita para produção

Decisão de Impacto: Split Temporal e Feature Tables Pré-computadas

Duas decisões de ML têm impacto directo na validade das métricas de avaliação e na ausência de *data leakage*:

Split temporal sem shuffle: as séries temporais energéticas têm forte dependência sequencial: o consumo às 14h de hoje está correlacionado com o consumo às 14h de ontem. Embaralhar os dados antes do split (abordagem comum para dados i.i.d.) introduz *data leakage*: o modelo vê dados do futuro durante o treino, inflacionando artificialmente as métricas de avaliação. O split temporal garante que o conjunto de teste representa dados genuinamente não vistos, com um horizonte temporal real de 6 meses.

Feature tables pré-computadas na Gold: lags (1h, 24h, 7d) e *rolling windows* (média 24h, máximo 7d) são calculados uma única vez em SQL Trino e armazenados como colunas da tabela Gold. A alternativa (calcular em Python durante cada run de treino) introduz risco de *training-serving skew*: se a implementação Python do lag diferir minimamente da implementação SQL em produção (e.g., tratamento diferente de nulos nas bordas da janela), as previsões do modelo degradam silenciosamente sem que as métricas de avaliação detectem o problema.

Relevância para o Negócio

Impacto Hipotético dos Três Data Products

DP-01: Monitorização de Défice Energético

Um comercializador que preveja défice com 1 hora de antecedência pode antecipar compras no mercado intradiário a preços mais favoráveis do que no mercado de balanço (tipicamente mais caro e com spreads de incerteza maiores). O modelo de classificação RandomForest com `target flag_defice (t+1h)` oferece directamente este sinal de decisão, com a `feature_importances` a revelar que as horas do dia e o saldo energético da hora anterior são os preditores mais relevantes.

DP-02: Estimativa de Custo Horário e Previsão de Carga

A feature table `feat_load_forecasting_hourly` com lags 1h/24h e *rolling window* 24h permite ao modelo GradientBoosting prever o consumo da próxima hora com base em padrões históricos recentes. Num cenário real, esta previsão alimentaria um sistema de optimização de portefólio: saber se o consumo sobe ou desce na próxima hora determina se é vantajoso comprar antecipadamente no mercado *day-ahead* ou aguardar pelo mercado intradiário.

DP-03: Previsão Meteo → Produção → Preço

A cadeia de modelos (Modelo A: produção renovável a partir de meteorologia; Modelo B: preço *spot* a partir de produção + meteorologia) reflecte uma hipótese de negócio verificável: em mercados com elevada penetração renovável como o ibérico, a produção fotovoltaica e eólica é o principal determinante marginal do preço *spot*. Um modelo que antecipe o preço a partir da previsão meteorológica (disponível 7 dias antes via Open-Meteo) permite estratégias de *hedging* com muito maior antecedência do que os modelos baseados em dados de mercado actuais.

Valor da Qualidade como Diferenciador

Os 110 checks de qualidade e os dashboards de observabilidade não são apenas boas práticas técnicas, tendo impacto directo em decisões de negócio. Um sinal de défice gerado a partir de dados com duplicados ou valores fora de range pode levar a uma decisão de compra desnecessária no mercado intradiário, com custo financeiro real e imediato. A rastreabilidade dados ↔ modelo (via MLflow tags e params) permite auditar *a posteriori* qualquer decisão automatizada que tenha usado um modelo específico, respondendo à pergunta “com que dados foi treinado o modelo que recomendou esta compra?”.

Conclusão

O projecto MIBEL implementou integralmente os seis critérios de avaliação, construindo uma plataforma de dados end-to-end funcional e operacional:

- **A)** Três Data Products especificados com contratos formais, SLAs calibrados com base nas características reais das fontes, e estratégia de versionamento semântico com `feature_set_version` obrigatório no DP-03
- **B)** Lakehouse local completo com arquitetura medallion Bronze/Silver/Gold, separação deliberada Hive (Bronze) vs. Iceberg (Silver/Gold), query engine Trino com federação multi-catálogo sobre MinIO
- **C)** Pipelines idempotentes com quality gates bloqueantes, reprocessamento incremental via DELETE+INSERT atómico, orquestração Flyte para o DP-02 Static, e no DP-02 Streaming: auditoria completa em `iceberg.audit`, retry com backoff exponencial, compactação Iceberg automática e SLA monitorizado
- **D)** Quatro dashboards Grafana provisionados como código JSON em git, com SQL directo sobre Trino e sem camadas de serving intermédias que possam introduzir *skew*

- **E)** 110 checks de qualidade distribuídos por Bronze, Silver e Gold de 4 pipelines; quality gate bloqueante em FAIL; tabelas `iceberg.audit` com lineage materializado por execução; distinção documentada entre WARNs esperados e FAILs críticos; demonstração controlada de injeção de dados sujos no DP-03 (nulos, fora de intervalo, duplicados, timestamps mal alinhados) com validação do mecanismo de limpeza Silver e da proteção da Gold via `_quality_flag = 'ok'`
- **F)** 5 modelos ML com rastreabilidade completa de dados via MLflow, split temporal sem *data leakage*, feature tables pré-computadas na Gold para eliminar *training-serving skew*, e Model Registry com 5 modelos registados e comparáveis

As decisões de maior impacto foram: separação Hive/Iceberg por camada, quality gate bloqueante em FAIL, Grafana sobre Trino como única camada de serving, split temporal nos modelos ML, feature tables pré-computadas na Gold e auditoria `iceberg.audit` com lineage automático no DP-02 Streaming. Todas estas decisões garantem que dados, modelos e execuções em produção reflectem fielmente a realidade do mercado energético português e são auditáveis *a posteriori*, um requisito fundamental em qualquer sistema de suporte à decisão com implicações financeiras.